

Application of signal separation to diffraction image compression and serial crystallography

JÉRÔME KIEFFER,^{a*} JULIEN ORLANS,^a NICOLAS COQUELLE,^a

SAMUEL DEBIONNE,^a SHIBOM BASU,^b ALEJANDRO HOMES,^a GIANLUCA SANTONI^a

AND DANIELE DE SANCTIS^a

^a*European Synchrotron Radiation Facility; 71, avenue des Martyrs; CS 40220; 38043 Grenoble Cedex 9 France, and* ^b*EMBL Grenoble; 71 avenue des Martyrs; CS 90181; 38042 Grenoble Cedex 9; France. E-mail: jerome.kieffer@esrf.fr*

background extraction; serial crystallography; peak-picking; lossy compression

Abstract

We present here a real-time analysis of diffraction images acquired at high frame-rate (925 Hz) and its application to macromolecular serial crystallography. The software uses a new signal separation algorithm, able to distinguish the amorphous (or powder diffraction) component from the diffraction signal originating from single crystals. It relies on the ability to work efficiently in azimuthal space and derives from the work performed on pyFAI, the fast azimuthal integration library. Two applications are built upon this separation algorithm: a lossy compression algorithm and a peak-picking algorithm; the performances of both is assessed by comparing data quality after reduction with XDS and CrystFEL.

1. Introduction

X-ray macromolecular crystallography is one of the most successful methods to determine the atomic structure of biological molecules. The achievable diffraction quality may often be limited by radiation damage. Although cryogenic conditions permit to extend the lifetime of crystals in the X-ray beam and to increase the maximum absorbed dose before inducing damage, they may hinder physiologically relevant conformations. This limitation has renovated the interest for room temperature macromolecular crystallography by applying a more drastic approach to overcome the radiation damage problem by collecting data from thousands of small crystals, in what became known as serial crystallography. First developed at X-ray free electron laser sources (XFEL) (Chapman *et al.*, 2011; Boutet *et al.*, 2012) the method is currently applied also at synchrotron sources (Diederichs & Wang, 2017; Gati *et al.*, 2014; Nogly *et al.*, 2015; Owen *et al.*, 2017).

1.1. Serial crystallography using synchrotron sources (SSX)

Serial crystallography consists in exposing thousands of small crystals to the X-ray beam only once in a serial way. Diffraction is collected with a very high flux density, in order to extract the most information from a single shot. This is in contrast with traditional rotational crystallography, where a complete dataset is collected from a single crystal rotated around one (or several) axis. Those SSX images represent a slice through the reciprocal space, thus intersect a lower fraction of the reciprocal space as the crystal is still, in comparison with rotational crystallography. To achieve a complete dataset, thousands of frames have to be collected, individually indexed and then merged. The high flux needed to collect all diffraction signal from a single crystal within a single exposure makes the SSX technique a good candidate to benefit from the 4th generation synchrotron sources, such as the new ESRF-EBS update (Chaize

et al., 2018). However, macromolecular crystallography beam-lines (MX) have been extremely specialized towards rotational data-collection and thus require modifications to the experimental setup to perform SSX experiments. The synchrotron serial crystallography beamline (ID29 at the ESRF) has been built to have a dedicated environment to perform SSX experiments with a high flux (using a larger energy bandwidth with a multilayer monochromator), a high-speed chopper (to separate X-ray pulses), several sample delivery methods and a fast detector.

1.2. JUNGFRÄU 4M detector

Macro-molecular crystallography has enormously progressed in the last decades with to the introduction of photon-counting detectors (Broennimann *et al.*, 2006). With their absence of read-out noise and their fast speed, the main limitation of photon-counting detectors is the achievable count-rate, i.e. how fast the electronic of a pixel is able to count arriving photons. Unlike photon-counting detectors such as the Eiger detector (Casanas *et al.*, 2016), the JUNGFRÄU detector (Mozzanica *et al.*, 2016) is an integrating detector, thus, it is not limited by the count rate, even under the very intense flux expected when recording Bragg-peaks. To cope with this photons density, every single pixel implements an automatic gain switching mechanism (3 levels) which offers a precision of the order of one third of a keV on the higher gain mode, a precision of the order of one photon in the intermediate level and the ability to cope with thousands of photons in the lower gain mode. Moreover, the JUNGFRÄU is able to operate at two kilohertz, which means the image must be read every half millisecond. Since the JUNGFRÄU detector is an integrating detector, dark current and flat-field correction have to be applied: every pixel has three, so called, pedestals and three gain values (one for each gain level). This large number of parameters per pixel makes the pre-processing of raw signal challenging at full speed: the signal from a single

pixel, initially packed with 16 bits per pixel, gets expanded to 32 bits (floating point or integer value), doubling the required bandwidth and the storage size (Leonarski *et al.*, 2020). The ID29 beamline features a JUNGFR AU 4M detector, operating at 1kHz, pace imposed by the chopper and synchronized with the photon bunches from the ESRF. At nominal speed, the detector will produce 16 GBytes of pre-processed data per second, making the data analysis and storage extremely challenging.

1.3. Requirements for online data processing

Serial crystallography is one of the techniques where online data processing is likely to have most impact: millions of images are to be collected, and existing detector already saturate the fastest storage systems, not even considering the cost of this storage. Beside this, only a few percentage of the frames are expected to contain diffraction signal and out of them, a fraction will be indexed, integrated and thus useful to solve the protein structure. Efficient processing of raw images is thus essential for SSX, with 4 levels of increasing complexity:

- image reconstruction with pedestal correction
- *veto*-algorithm: be able to sieve-out images with poor signal
- save only pixels with diffraction signal
- precise location of peak position with indexation (Gasparotto *et al.*, 2024)
- real-time integration of diffraction peaks

.

Reconstruction and pedestal correction have been already described in Debionne *et al.* (2022a). This document will focus on the subsequent steps: the detection of signal is addressed in section 2, sparse data compression in section 3, and peak-finding in section 4 before drawing some conclusions in section 5.

2. Algorithm for the separation of the amorphous background from the Bragg peaks

2.1. Background scattering

The simplest implementation of Bragg peaks separation is to assume that the background signal originates from scattering of amorphous material giving an isotropic signal that ideally presents only smooth variations. Before background subtraction, the raw signal has to be corrected for dark noise and for any systematic anisotropic effects such as polarization corrections. Unlike X-FEL, synchrotron X-ray beam is better characterized in energy and shows little to none pulse to pulse variability. All anisotropic correction can be easily modelled and taken into account. The same method can be extended to separate Bragg peaks from powder diffraction if the powder signal is isotropic, i.e. without preferred orientation.

The initial implementation of signal separation in pyFAI (Kieffer & Wright, 2013) was relying on a 2D polar transform followed by a median filter in the azimuthal dimension to calculate the amorphous scattering curve. Although this method has been successfully used for large dataset analysis (Bordet *et al.*, 2021), it presents four major drawbacks:

- The 2D averaging mixes the signal originating from several pixels and blurs the signal.
- Pixel-splitting is needed to leverage the Moiré effect in the 2D averaging, but this further increases the blurring (Fang *et al.*, 2020).
- The 1D curve obtained after the application of the median filter shows sharp jumps from one azimuthal bin to its neighbour.
- The median filter is computationally heavy since it is required to sort out every azimuthal bin.

We improved on this by developing a new, efficient way of performing the azimuthal

averaging (including the associated uncertainty propagation).

2.2. Efficient azimuthal averaging and uncertainties evaluation

2.2.1. Pre-processing: The first step of the analysis consists in applying a pixel-wise correction for dark current and several normalization corrections (Kieffer *et al.*, 2020):

$$I_{cor} = \frac{signal}{norm} = \frac{I_{raw} - I_{dark}}{F \cdot \Omega \cdot P \cdot A \cdot I_0} \quad (1)$$

In equation 1, the numerator (referred as *signal* hereafter) is the subtraction of the dark current I_{dark} from the the detector's raw signal I_{raw} . The denominator (hereafter *norm*) is a normalization factor composed of the product of F : a factor accounting for the flat-field correction, Ω : the solid angle subtended by a given pixel, P : the polarisation correction term and A : the detector's apparent efficiency due to the incidence angle of the photon on the detector plane. For integrating detectors, photons with high energy see longer sensor path with larger incidence angles compared to the normal thickness, thus they have a higher detection probability. Intensity is also normalized by the incoming flux I_0 , but being it independent from the pixel position, this correction can be applied when convenient.

2.2.2. Azimuthal averaging: Historically, the azimuthal averaging was implemented using histograms (Hammersley *et al.*, 1996). Since the geometry of the experimental setup is fixed during the acquisition, a look-up table listing all pixels contributing to each azimuthal-bin can be built and used to speed-up calculations (Kieffer & Ashiotis, 2014). The azimuthal transformation being a linear transformation, it can be implemented as a matrix multiplication, with a sparse-matrix representing the transformation and a dense vector containing the flattened view of the diffraction image. The compressed sparse row (CSR) matrix representation is preferred for its efficiency in performing dot products with dense vectors (Toledo, 1997). The coefficients $c_{i,r}$ of

the matrix are the fraction of area of a pixel i falling into the radial bin r . In the case where pixel splitting is deactivated these coefficients ($c_{i,r}$) are always one (and zero elsewhere) since each pixel contributes to a single bin. The sparse matrix multiplication can be used to sum efficiently values for all pixels belonging to the same bin. The summed signal divided by the summed normalization provides the weight-averaged intensity over all pixels falling in the bin at the distance r from the center, as formalized in equation 2:

$$\langle I \rangle_r = \frac{\sum_{i \in \text{bin}_r} c_{i,r} \cdot \text{signal}_i}{\sum_{i \in \text{bin}_r} c_{i,r} \cdot \text{norm}_i} \quad (2)$$

2.2.3. Uncertainty evaluation from Poisson distribution. Photon counting detectors, such as Eiger detectors, suffer from hardly any error beside the counting uncertainty which is often referred to as Poisson statistics. This statistical law is described by a single parameter λ which is related to the mean μ and standard deviation σ from a normal distribution by: $\lambda = \mu = \sigma^2$. Other sources of noise, like the dark current noise in the case of an integrating detector, superimpose quadratically on the Poisson noise for integrating detectors, as presented in equation 3:

$$\text{var}_I = (\sigma_I)^2 = \langle I_{\text{raw}} \rangle + (\sigma_{\text{dark}})^2 \quad (3)$$

During the azimuthal integration, the coefficient of the sparse matrix needs to be squared at the numerator when propagating the variance (equation 4) to have uncertainties σ proportional to the fraction of the pixel considered.

$$(\sigma_r(I))^2 = \frac{\sum_{i \in \text{bin}_r} c_{i,r}^2 \cdot \sigma_i^2}{\sum_{i \in \text{bin}_r} c_{i,r} \cdot \text{norm}_i} \quad (4)$$

One should distinguish the *uncertainty of the mean* (sometimes referred to as the standard error of the mean, *sem*), which describes the precision with which the mean is known (and described in Kieffer *et al.* (2020)), from the *uncertainty of the pixel*

value (often referred to as standard deviation, *std*) which describes the uncertainty with which the pixel value is known. Those two value differ only by the square root of the number of measurements in the case of an arithmetic mean: $sem = std/\sqrt{N}$ with N being the number of pixel contributing to the bin. When considering the weighted average, the previous formula becomes:

$$sem_r = std_r \frac{\sqrt{\sum_{i \in bin_r} c_{i,r}^2 \cdot norm_i^2}}{\sum_{i \in bin_r} c_{i,r} \cdot norm_i} \quad (5)$$

Thus, the more data points are collected, the more precisely the mean value is known but the uncertainty for a given point remains the same. Since this document focuses on the uncertainties of pixel values, the *standard deviation* will systematically be used from here on.

2.2.4. Uncertainty evaluation from the variance in a bin. Unlike photon counting detectors, most detectors do not follow the Poisson distribution and therefore the definition of a relation $\sigma^2 = f(I)$ is not simple, if possible at all. The integrating JUNGFR AU detector has a complex gain switching mechanism (Leonarski *et al.*, 2020) which makes this equation complicated. A generic approach is proposed to measure the variance in every single azimuthal bin:

When considering the diffraction of an isotropic compound (liquid, amorphous or perfect powder), all pixels contributing to the same radial bin should see the same flux of photons (after correction of anisotropy like polarization) and the deviation to their intensities can be used to estimate the uncertainty. This approach is of course limited when considering the signal coming from few large crystalites (where rings becomes spotty) but it provides an upper bound for the uncertainty. Variances (thus standard deviations) are usually obtained in a two steps procedure: one pass to calculate the average value (equation 2) and a second to calculate the deviation to the average

(equation 6). This double pass approach can be implemented using sparse matrix multiplication. It requires twice the access to each pixel value, and extra storage space, but it is numerically robust (i.e. not prone to numerical-error accumulation).

$$(\sigma_r(I))^2 = \frac{\sum_{i \in \text{bin}_r} c_{i,r}^2 \cdot \text{norm}_i^2 \cdot (\frac{\text{signal}_i}{\text{norm}_i} - \langle I \rangle_r)^2}{\sum_{i \in \text{bin}_r} c_{i,r}^2 \cdot \text{norm}_i^2} \quad (6)$$

and:

$$(\sigma_r(\langle I \rangle))^2 = \frac{\sum_{i \in \text{bin}_r} c_{i,r}^2 \cdot \text{norm}_i^2 \cdot (\frac{\text{signal}_i}{\text{norm}_i} - \langle I \rangle_r)^2}{(\sum_{i \in \text{bin}_r} c_{i,r} \cdot \text{norm}_i)^2} \quad (7)$$

Single pass implementations of variance calculation are faster than double pass ones since they access pixels only once and offer, in addition, the ability to perform parallel reductions (Blelloch, 1996), i.e. work with blocks of pixels. Schubert & Gertz (2018) present a complete review on the topic, which introduces a formalism adapted here for crystallography. Assume the weight for a pixel being $\omega_i = c_i \cdot \text{norm}_i$. If P is a partition of the ensemble of pixels falling into a given azimuthal bin, let Ω_P , V_P and VV_P be the sum of weights (Eq 8), the weighted sum of V (Eq. 10) and the weighted sum of deviation squared (Eq 11) over the partition P :

$$\Omega_P = \sum_{i \in P} \omega_i = \sum_{i \in P} c_i \cdot \text{norm}_i \quad (8)$$

$$\Omega\Omega_P = \sum_{i \in P} \omega_i^2 = \sum_{i \in P} c_i^2 \cdot \text{norm}_i^2 \quad (9)$$

$$V_P = \sum_{i \in P} \omega_i \cdot v_i = \sum_{i \in P} c_i \cdot \text{signal}_i \quad (10)$$

$$VV_P = \sum_{i \in P} \omega_i^2 \cdot (v_i - V_P/\Omega_P)^2 \quad (11)$$

The weighted average and associated variances are then expressed as:

$$\langle I \rangle_P = \frac{V_P}{\Omega_P} = \frac{\sum_{i \in P} c_i \cdot \text{signal}_i}{\sum_{i \in P} c_i \cdot \text{norm}_i} \quad (12)$$

$$\text{std}^2 = (\sigma_P(I))^2 = \frac{VV_P}{\Omega\Omega_P} \quad (13)$$

$$sem^2 = (\sigma_P(< I >))^2 = \frac{VV_P}{\Omega_P^2} \quad (14)$$

In this formalism, equations 2 and 12 on one side and equations 6 and 13 are actually equivalent. Schubert & Gertz (2018) present the way to perform the union of two sub-partitions A and B of a larger ensemble which opens the doors to parallel reductions, which are especially efficient when implemented on GPU:

$$\Omega_{A \cup B} = \Omega_A + \Omega_B \quad (15)$$

$$V_{A \cup B} = V_A + V_B \quad (16)$$

$$VV_{A \cup b} = VV_A + \omega_b^2 \left(v_b - \frac{V_A}{\Omega_A} \right) \left(v_b - \frac{V_{A \cup b}}{\Omega_{A \cup b}} \right) \quad (17)$$

$$VV_{A \cup B} \approx VV_A + VV_B + \frac{\Omega \Omega_B (V_A \cdot \Omega_B - V_B \cdot \Omega_A)^2}{(\Omega_{A \cup B} \cdot \Omega_A \cdot \Omega_B^2)} \quad (18)$$

While equations 15 and 16 are trivial, equation 17 describes the nominator of the variance of an ensemble when adding an extra member b to the A . Unfortunately, the slight difference of formalism between Schubert & Gertz (2018) and this work prevent some simplification to occur and leads to the approximate numerator of the variance (VV) in the case of the union of two ensemble A and B (eq. 18), used in OpenCL reduction ¹. A numerical stability issue can arise from it when V_A or V_B are very small and this issue is addressed by using double-precision arithmetic when implemented on CPU and double-word arithmetic when running on GPU (Joldes *et al.*, 2017).

2.2.5. Comparison of uncertainty models: Figure 1 (right hand side) presents the uncertainties (for the pixel value) as calculated from a background frame with pure Poisson noise (displayed on the left hand-side, synthetic data) using the two algorithms previously described: the Poisson model or calculated from the variance in the azimuthal bin. While the two curves show similar amplitude, except in the corner of the

¹ see <https://github.com/silx-kit/pyFAI/blob/main/doc/source/usage/tutorial/Variance/uncertainties.ipynb>

detector where very few pixels contribute to each of the azimuthal bin, the variability of the “azimuthal” model is much more important from one bin to the neighboring one.

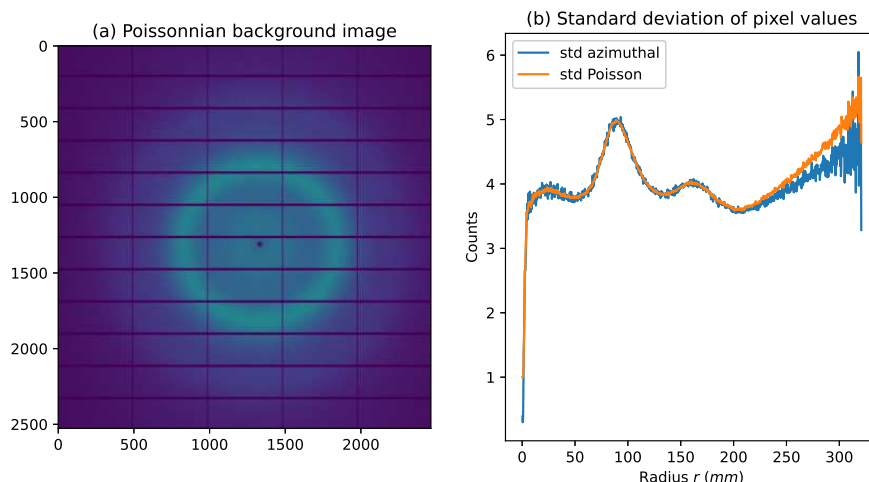


Fig. 1. (a) Simulated diffraction frame with pure azimuthal Poisson noise of a Pilatus 6M detector and (b) uncertainties for pixel intensity as measured with the distance to the mean (azimuthal-model, blue) or from the Poisson model (orange).

2.3. Histogram intensity

The figure 2 presents the diffraction from a single crystal of insulin collected with a Pilatus 6M detector (Fig 2a) and several curves obtained from azimuthal integration of those data: figure 2b is the azimuthally integrated signal (blue curve) where Bragg peaks are seen as spikes on top of a smooth background. Figure 2c presents the uncertainties measured according to the Poisson distribution (orange curve) or the deviation in the ring (blue curve). The later presents much larger values since Bragg peaks contribute a lot to the deviation despite they represent few pixels: this highlights the sensitivity of the mean/std to outliers. In the figure 2d are presented histograms of pixel intensity for pixels laying at 87mm and 160mm from the beam center. Each of those histograms is composed of a bell-shaped distribution with few positive outliers which are usually Bragg peaks. Those histograms in figure 2d have been fitted with a

Gaussian curve and both the center (μ) and the width (σ) of the curve match roughly with the average (in 2b) and uncertainties (in 2c).

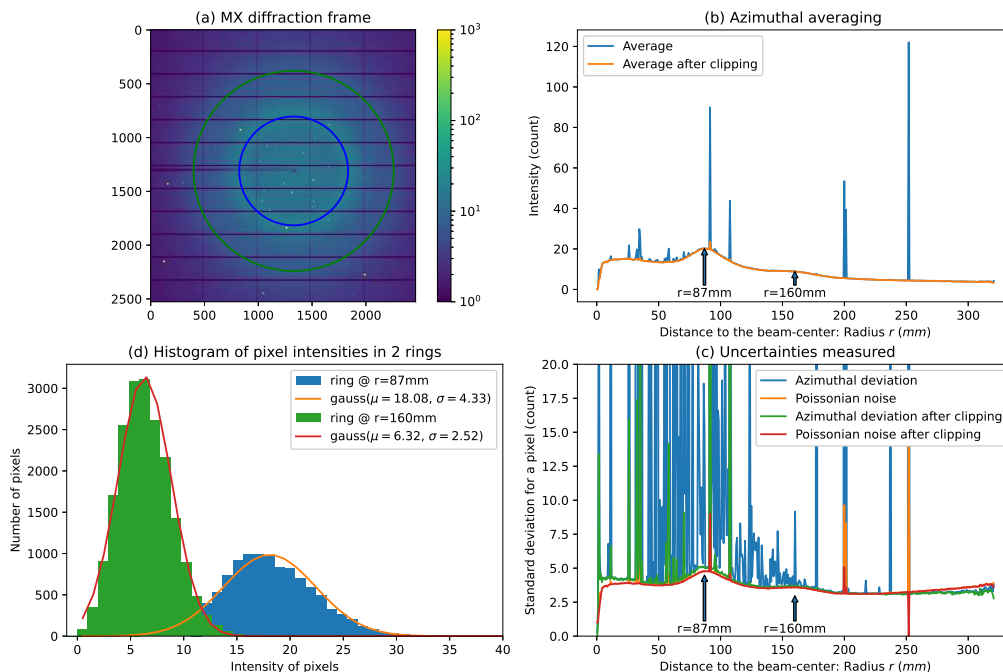


Fig. 2. Single crystal diffraction frame obtained from insulin with a Pilatus 6M (a) with the azimuthally averaged signal (b), before and after clipping data. Uncertainties are presented in (c) when calculated assuming a Poissonian error model (orange, red) or when measuring the deviation within all pixels in a ring (green, blue). (d) Histogram of intensities for two rings at $r=87\text{mm}$ and $r=160\text{mm}$ from beam center with the distribution fitted as Gaussian curves: $g(x) \propto \exp(-\frac{(x-\mu)^2}{2\sigma^2})$.

The core idea of the algorithm for background extraction is to model the distribution of background pixels. Unlike Bayesian statistics (Sivia & Skilling, 2006) where the cost function is usually tuned to weight less outlier, here, those outliers are simply flagged and discarded. Positive outliers can reasonably be assigned to Bragg peaks and negative outliers to shadows or defective pixels. The distribution is recalculated after discarding pixels for which intensity differ from the average value by more than n times the standard deviation (Eq. 19), where n is called the Signal to Noise Ratio (SNR). This clipping re-centers the distribution of remaining pixels since the mean is

sensitive to outlier values:

$$|I - \langle I \rangle| > n \cdot \sigma(I) \quad (19)$$

The orange plot in figure 2b presents the average after having discarded those outliers, and the orange and green curve of figure 2c are the uncertainties calculated after this clipping. After clipping, the average and uncertainties curves have lost most of their spikes, which means that most Bragg peaks and shadowed pixel have been discarded.

Of course this works only if the background signal is isotropic, ideally smoothly varying, and there are many more background pixels than peaks or shadowed pixels. While shadows can be handled with a user defined mask, anisotropy in the background scattering, as it is sometimes observed with certain stretched plastic films in fixed-target mode, is much more challenging and will not be addressable with this analysis.

2.4. *Sigma-clipping*

The *sigma-clipping* algorithm consists in applying this outlier rejection several times. If the initial distribution is mono-modal, this algorithm enforces gradually the data to be sampled symmetrically around the maximum probability, which is likely to look like a normal distribution. If the initial distribution is more complicated (typically multi-modal), the necessary larger standard-deviation will prevent most outlier pixels from being rejected, making it more conservative. The sigma-clipping algorithm takes two parameters: the number of iterations and the rejection cut-off (SNR). Despite the execution time is proportional to the number of iteration of sigma-clipping, iterations should continue until no more outliers are found, so that the background data can be treated assuming a normal distribution. Since the loop exits as soon as no more outliers were discarded at the clipping step, having an arbitrary large number of iteration is not really an issue for the execution time and the number of actual iteration is usually few (3 is commonly observed).

2.4.1. Limits of the Poissonian approach: The evaluation of uncertainties based on the variance within a radial shell (azimuthal model) has been developed after numerical artifacts were discovered while performing sigma-clipping with a Poissonian approach. Some azimuthal bins were showing no pixel contribution at all and thus appeared without any mean nor uncertainties, jeopardizing the complete background extraction algorithm. This artifact was directly linked to the usage of Poisson statistics and can be demonstrated with a simple distribution of 2 pixels, having values 1 and 199. The mean of this distribution is 100 and the standard deviation is also close to 100 while the uncertainty derived from a Poissonian law would be close to 10 (i.e. $\sqrt{100}$). With the azimuthal error model, both pixels are a 1σ from the mean while with the Poissonian error-model, pixels are at 10σ . This explains why bins featuring strong Bragg-peaks on top low background got completely emptied from any contributing pixels when sigma-clipping was performed assuming Poissonian noise.

Unlike the Poisson error-model, the uncertainties obtained from the azimuthal-model are resilient to diffraction data coming from several types of samples but show much more variability from one bin to its neighbor (Fig. 1b). PyFAI introduces an *hybrid* error-model which uses the azimuthal error-model for the sigma-clipping stage which trims the ensemble of pixels to become mono-modale. The uncertainties are then calculated using a the Poisson error-model on the trimmed ensemble.

2.4.2. Clipping threshold can be automatically calculated based on a variation on Chauvenet's criterion (Maples *et al.*, 2018) where one would accept to discard only a single pixel in a ring with a signal already following a normal law. Thus, the threshold value is adapted to the *size* of the distribution, i.e. the number of pixels in each ring (Eq. 20), which can reach several thousands and shrinks with iterations. Typically

the numerical value for this cut-off varies from 2 to 4.

$$SNR_{chouv.} = \sqrt{2 \log\left(\frac{size}{\sqrt{2\pi}}\right)} \quad (20)$$

The worse case scenario for sigma-clipping corresponds to an initial distribution very far from a normal distribution, as the bimodal distribution seen in the previous paragraph. Another challenging situation occurs close to the detector corners where the background signal is low and the size of the distribution is decreasing. For example this cut-off parameter increases from 2.7 to 3.5 when the size of the ensemble increases from 100 to 1000 elements. So, for a Poissonian detector and a low count rate of one ($\lambda = \mu = \sigma^2 = 1$), any pixel with intensity greater than 4 is discarded with the ensemble of 100, greater than 5 for the ensemble of 1000 pixels.

3. Application to lossy image compression for X-ray diffraction

Diffraction images from protein crystals usually exhibit an isotropic background on top of which Bragg peaks appear (discarding any diffuse scattering). The sigma-clipping algorithm can be used to select the background level and more importantly the associated uncertainty. This lossy compression algorithm consists in saving only pixels which intensity is above the average background value (μ) plus n standard deviation (σ).

The decompression simply restores those intense pixels and builds a smooth background for the missing ones (possibly with noise). The cut-off value n (also called SNR_{pick}) controls the amount of data to store. It is linked to the compression ratio: assuming a normal distribution has been enforced at the sigma-clipping stage: 16% of the pixel are to be recorded with $n = 1$; 2.3% for $n = 2$ and only 0.13% for $n = 3$ as depicted in figure 3.

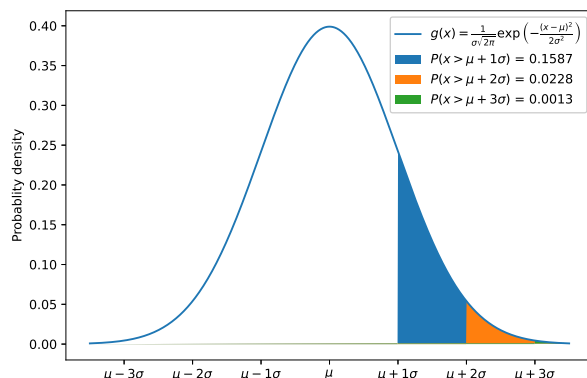


Fig. 3. Normal distribution and probability of having pixels with intensities above a certain threshold. The cut-off parameter governs how much signal is integrally kept, thus the achievable compression rate on the one hand and the limit of quality of data on the other. It is the user's responsibility to set wisely this threshold.

The compressed data-format consists in:

- Pixels intensities and positions for pixels worth saving.
- Average level of the background and associated uncertainties as function of the distance to the beam-center.
- The distance to the center of every pixel.

Since diffraction analysis software are performing some kind of noise-level analysis, the background signal has to be regenerated with intensity and noise similar to the original data.

3.1. Sparsification / Compression

The sigma-clipping algorithm was originally written for real-time sparsification of single crystal diffraction data and its integration into the LImA2 detector control system (Petitdemange *et al.*, 2018) for the JUNGFR AU 4M detector used at ESRF ID29 is described in (Debionne *et al.*, 2022b). The real-time constrain imposed to develop code running on GPU since those devices are several times faster than equivalently

priced processors. All algorithms were developed in OpenCL (Khronos, 2008) and implemented in the *pyFAI* software package (MIT-licence, available on github). A command line tool called ‘sparsify-Bragg’ is available for testing off-line since version 2023.01.

All the pixel coordinates and intensities are stored in a HDF5 container (The HDF Group, 2000-2021) following the NeXus convention (Könnecke *et al.*, 2015), together with a snippet of Python code explaining how to rebuild the dataset. All sparse datasets (averaged and uncertainties curves, pixel coordinates, etc..) are compressed with the bitshuffle-LZ4 (Masui *et al.*, 2015) lossless compression.

3.2. *Densification / Decompression*

Since no crystallographic software package can deal with this sparse format (yet), the densification code was developed to regenerate initial frames and the ‘densify-Bragg’ program was made available as part of the FabIO (Knudsen *et al.*, 2013) software package (MIT license). The source code is available on github and ‘densify-Bragg’ is available since version 2022.12, available via usual channels like pip or conda install. The software constraints for this densification code are very different from the one for sparsification since this code can be used by the final users. For this reason ‘densify-Bragg’ was optimized to run on multi-core CPU. Maybe an important consideration is whether, regardless from the file format, it is necessary to reconstruct the background or not. In fact, some crystallographic reduction program like CrysAlisPro (Rigaku Oxford Diffraction, 2015) provide a better result with noise-less background while XDS (Kabsch, 2010), which performs a deep noise analysis, needs to have the noisy background properly restored. Shaded regions are never reconstructed properly and should be masked adequately in the reduction software.

Future development will focus on a HDF5-plugins able to provide access to densified

images from their sparse representation using user-defined function in HDF5 (Real & de Bayser, 2021). This will allow any analysis software (already able to read HDF5 files) to treat sparse data as if they were dense, removing the burden of densifying the images from the users.

3.3. Performances on a single crystal protein dataset

The performances for a lossy compression algorithm are to be evaluated along many directions: compression and decompression speeds, compression ratio and degradation of the recorded signal. In the following example we present the sparsification of a Lysozyme (HEWL) dataset obtained using a traditional oscillation data-collection. Data were collected on an Eiger 4M detector (Dectris, 2014), selected for its similarity in size and performances with the JUNGFRÄU detector. Those data are then densified again to regenerate the data and processed in XDS (Kabsch, 2010). Data quality indicators are finally compared between the original dataset and the one which went through the lossy compression presented here.

3.3.1. Compression ratio After sparsification (picking cut-off: 2σ , error-model: Poissonian), the dataset still weights 103MB which represents a 15x compression ratio compared to the standard procedure. For conformance with the state of the art, the reference dataset was re-compressed using the bitshuffle-LZ4 algorithm (Masui *et al.*, 2015), for which the 1800 frames weight 1500MB (instead of the 5000GB of the original files compressed in LZ4).

The maximum theoretical compression ratio for 2σ is 22x (figure 3, neglecting the storage of the background data and effects of the lossless compression). To evaluate the effective maximal compression ratio, the dataset was median-filtered along the image stack to produce an image without peaks. A background dataset of 1800 such images

sparsifies into a 11MB HDF5 file which represents a compression ratio of 136x ! Indeed, only 19 pixels were saved per frame and the compressed numerical values are mostly the same, which compresses great with bitshuffle-LZ4.

For ESRF-ID29, where the JUNGFR AU 4M can operate close to 1 kHz, the pedestal+gain pre-processing convert 16-bits integers into 32-bits floating point values, doubling the bandwidth for data saving. The detector outputs the data via 8×10 Gbit/s network links and the storage is performed via a single 25 Gbit/s link, making a minimum compression ratio of 6.4x.

In production conditions, ID29 users have to trade between:

- Detector speed: operate at lower speed (231 Hz) to be able to save the pre-processed data.
- Energy resolution: floating data compress badly, so they are better stored as integer, where the number of ADU per photon can be tuned. With 1 ADU per photon, one gets the best compression rates but one loses the sub-photon energy resolution of the JUNGFR AU detector.
- Use the burst mode: acquire only shorter datasets and make pauses to let the data reach the disk in the central storage.
- Discard pixel of lower intensity: this is a new possibility offered by this algorithm.

A cutoff at 1.5σ should already providing the 6.4x compression ratio needed (Fig. 3).

3.3.2. Compression speed: The compression speed has been measured on the computer designed for online data-reduction of the JUNGFR AU detector (Debionne *et al.*, 2022b): an IBM AC922 using two Power9 processors and two Nvidia Tesla V100 GPU. The sequential execution of the code on the GPU takes about 4 ms to process one image and uses one single CPU-core and a GPU. In production conditions, two such computers are

diving each two GPUs, allowing to use the detector at its nominal speed close to 1kHz. The main bottleneck remains the networked saving of the different files: preprocessed files, sparse files, peak positions, accumulated frames, ... not all can be saved at all time when operating at full speed.

3.3.3. Decompression speed: The decompression of those data should typically be performed on a standard workstation (here running two Intel Xeon Gold 6134 CPU @ 3.20GHz): the reconstruction speed is found to take 30 s for the full dataset, while writing of the densified dataset (with bitshuffle-LZ4 compression) takes 45s. Densification is thus faster than writing on disk. The reading time of the input sparse dataset is negligible ($<2s$).

3.3.4. Quality of the restored dataset: The densified dataset was processed via XDS and the summary indicator for the quality of the results are compared with the one coming from the reduction of the original dataset. Since those integrator are measured on integral peaks with $I/\sigma > 3$ and the sparsification was performed with a cut-off of 2, those result should be almost unaffected, which is confirmed in the table 1.

Table 1. *Quality indicators after peak integration and averaging using XDS (Kabsch, 2010).*

Lysozyme (HEWL) dataset provided by Dectris for advertising their Eiger-4M detector

(Dectris, 2014).

Indicator	Initial dataset			Lossy compressed dataset (2σ)		
	2.91Å	2.06Å	all	2.91Å	2.06Å	all
Completeness	98.8	90.8	93.8%	99.8	90.6	93.5%
$R_{obs} = \frac{\sum I_{h,i} - I_i }{\sum I_{h,i}}$	9.9	57.3	12.5%	9.2	61.2	11.4%
$R_{expected}$	8.8	73.2	15.0%	8.2	68.7	12.1%
R_{meas} (Diederichs & Karplus, 1997)	10.3	61.2	13.2%	9.6	65.5	12.0%
$CC_{1/2}$ (Karplus & Diederichs, 2012)	99.7	94.0	99.7	99.6	95.4	99.7
$< I/\sigma >$	25.80	5.39	10.52	26.33	4.09	10.17

Of course those data were collected on a test sample with very intense signal, but it demonstrates the algorithm does not destroy the signal. But with more challenging samples, exhibiting lower I/σ , the threshold for picking pixels has to be lower to

ensure all pixels relevant for subsequent analysis are actually preserved. Unless, and as described in Galchenkova *et al.* (2024), this sparsification would be detrimental for the quality of the reduced data.

3.4. Influence of the sparsification on the quality of serial-crystallography data acquired with Eiger detector

Tiny crystals of egg white lysozyme (HEWL) + Gadolinium were deposited on a SiN membrane and this membrane was scanned on the ID30A3 beamline at the ESRF (MASSIF3) using a Eiger 4M detector. The dataset consists of 11637 frames of images, out of which 11512 were properly indexed.

3.4.1. Quality indicator degradation: The R_{free}/R_{work} quality indicators (Brünger, 1992) were calculated for the initial dataset and compared with the same dataset sparsified at 0.8 , 1.0 , 1.4 and 2.0σ and subsequently re-densified (figure 4).

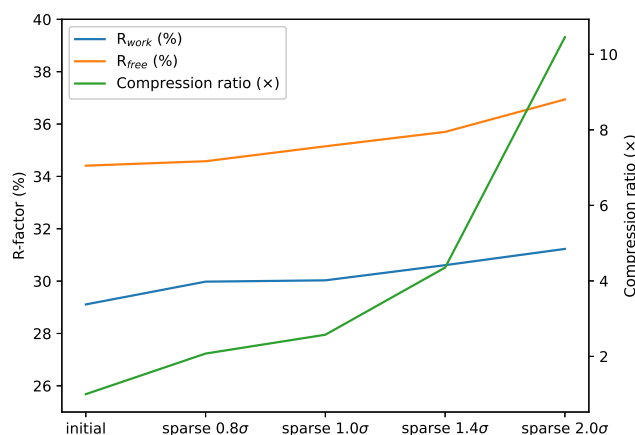


Fig. 4. Degradation of R_{work} and R_{free} crystallographic quality indicators on the integral dataset of HEWL+Ga (11k frames) and actual compression rates, when increasing the levels of sparsification.

As expected, the crystallographic quality indicator show a gradual degradation with

increasing sparsification but R_{free} shows little degradation with little sparsification (0.8σ , compression ratio of $2\times$).

3.4.2. Quality indicators as function of the resolution shell: The same R_{work} and R_{free} quality indicators are reported in figure 5 as function of the resolution shell in order to monitor if the degradation is uniform among shells or if it affects mostly the outer shells. Those indicators compare the initial dataset with the sparsified ones at 0.8σ and 2.0σ which were subsequently re-densified for the analysis.

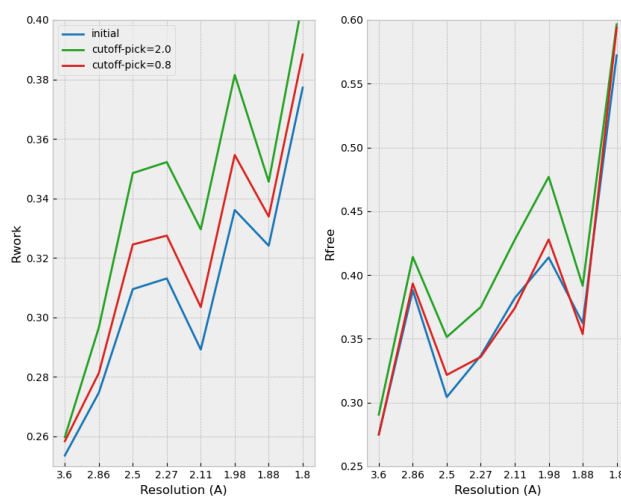


Fig. 5. R_{work} and R_{free} crystallographic quality indicators at different resolution shells obtained from the complete dataset (11k frames) of HEWL+Ga. The quality of the sparsified data (at 0.8σ and 2.0σ) are compared with the initial dataset.

Both R_{work} and R_{free} exhibit degradation as expected and this degradation is rather uniform over all shells, it does not affect more the outer-shells. The degradation of R_{free} at moderate sparsification (0.8σ) is once again very limited.

3.4.3. Ability to phase sparse data and quality of the density map: One of the main concern with sparsification is that it may degrade the weak anomalous signal which is

precious for phasing. The HEWL+Ga dataset was truncated at different length (from 3000 to 7000 out of the 11512 frames) in order to "artificially" decrease the anomalous signal strength in the dataset. The figure 6 shows the number of residues which were properly placed in the electron density map with an automatic procedure.

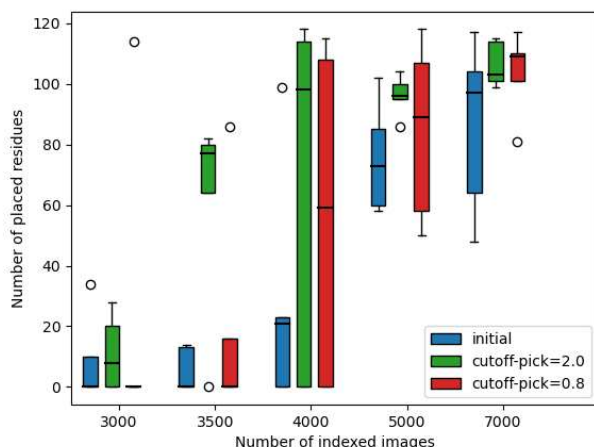


Fig. 6. Influence of the sparsification (at 0.8σ and 2.0σ vs initial dataset) on the ability to phase a the HEWL+Ga protein with an artificially reduced number of frames, in order to limit the strength of the anomalous signal.

The sparsified dataset does not show less residues properly placed after the procedure, it even looks marginally better than the initial dataset, especially for larger compression factor where the dataset could be phased with as little as 3500 frames. With Eiger detector data, a sparsification at 0.8σ offers a nice $2\times$ extra compression without noticeable degradation of the quality of the processed data.

3.5. Influence of the sparsification on the quality of serial-crystallography data acquired with JUNGFR AU detector

3.5.1. Dataset description: Small crystals of NQO1 sample (Grieco *et al.*, 2024), complexed with NADH, were collected at ESRF-ID29 using a fixed-target sample environment and the JUNGFR AU 4M detector (PDB: 8RFM). The complete dataset represents

574k frames (5 TB) out of which 25809 frames were selected with nice peaks (4.5% of the total). The indexation rate for dense data, and two sparse dataset (cut-off at 1.0σ and 1.4σ) are reported in table 2 and the crystallographic quality indicators are reported in the figure 7 in reciprocal space and figure 8 in real space.

Table 2. *Indexation statistics of the NQO1 dataset (25809 frames, 4.5% of the complete dataset)*

	dense	sparse 1.0σ	sparse 1.4σ
Disk space	166 MB	52 MB	32 MB
Compression	$1\times$	$3.2\times$	$5.2\times$
Frames indexed	22874	20176	20384
Indexation rate	88.6%	78.2%	79.0%

3.5.2. Data statistics: The processing was performed with *CrystFEL* (White *et al.*, 2012) (version 0.11) with the geometry of the detector optimized with the *millipede* procedure (Blobel & Kleinwort, 2002; Kleinwort, 2021-2024). Indexing was performed with the default parameters of *xgandalf* (Gevorkov *et al.*, 2019) based on peak-position by *peakfinder8* (Barty *et al.*, 2014).

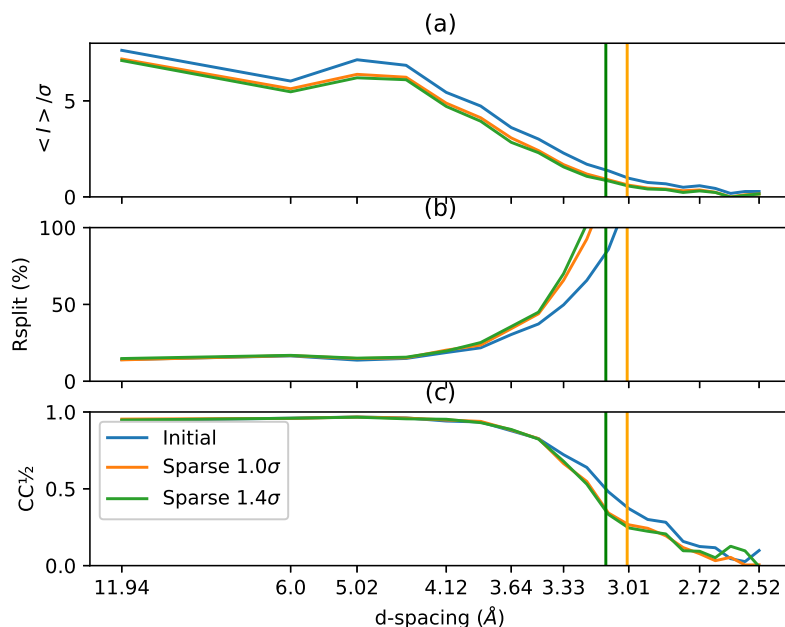


Fig. 7. Comparison of crystallographic quality indicator as function of the resolution shell for sparsified data (1.0σ in orange and 1.4σ in green) in comparison to the initial dense data (in blue): (a) signal/noise ratio, (b) R_{split} and (c) $CC_{1/2}$. The two vertical lines indicate the resolution shell at which the signal/noise ratio drops below the sparsification threshold, i.e. the limit at which all signal is expected to be lost.

From the $\langle I \rangle / \sigma$ curve, figure 7a, one can see there is a systematically lower SNR from sparsified data in comparison with the initial dataset, regardless to the resolution shell. This could be due to a too large amount of noise added when densifying the data, especially that this dataset was acquired (and processed) at the full energy resolution of the JUNGFRÄU detector, here 485 ADU/photon (in comparison, an Eiger has 1 ADU/photon). The SNR curve obtained on uncompressed data was used to assess the resolution shell at which there is no more signal expected to be saved, those are the two vertical lines in green (1.4σ) and orange (1.0σ). On the right of those lines, there is supposed to be no more signal, while on the left, there is supposed to be no degradation of the signal in an ideal case. The evolution of R_{split} and $CC_{1/2}$, plotted

in figure 7 b and c, show a degradation of the quality which is much earlier, by half an Angstrom, in comparison with what would have been expected.

3.5.3. Refinement statistics: Those data were finally refined in real-space using *phenix.refine* (Afonine *et al.*, 2012) until a resolution of 2.7Å, which is noticeably better than what can be expected from the sparse data (limited to 3.0Å and 3.1Å for 1.0 σ and 1.4 σ , respectively). The results are summarized in figure 8.

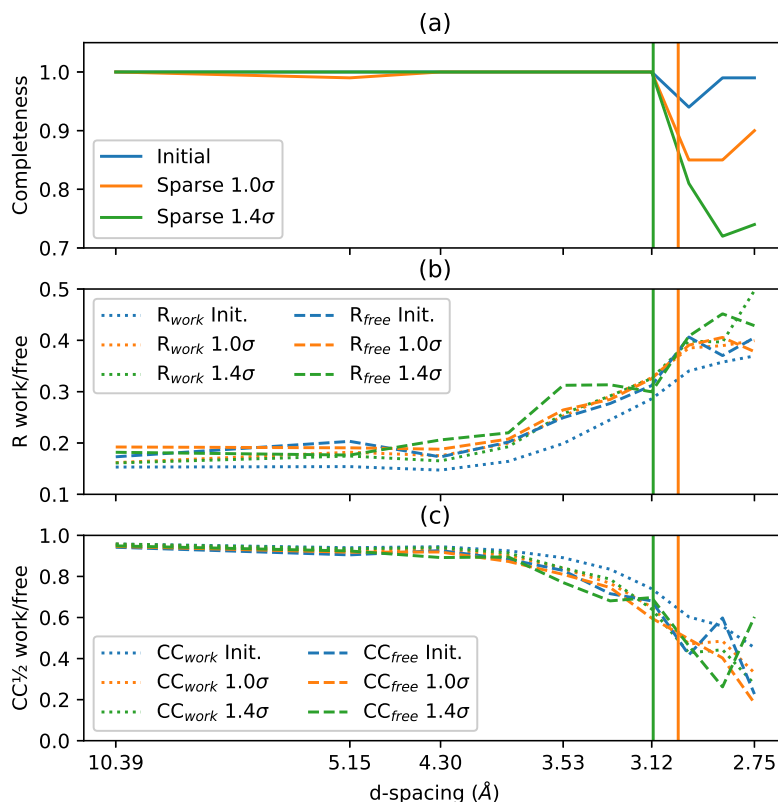


Fig. 8. Comparison of crystallographic quality indicator in real space after refinement using *phenix.refine* as function of the resolution shell for sparsified data (1.0σ in orange and 1.4σ in green) in comparison to the initial dense data (in blue): (a) Completeness of the dataset, (b) R_{work} (dotted) and R_{free} (dashed) and (c) $CC_{1/2}^{work}$ (dotted) and $CC_{1/2}^{free}$. The two vertical lines indicate the resolution shell at which the signal/noise ratio drops below the sparsification threshold, i.e. the limit at which all signal is expected to be lost.

The figure 8a contains the completeness of the dataset as function of the resolution shell. It confirms the sparsification algorithm works as expected, and that the signal starts to degrade where the SNR drops below the picking threshold. The figure 8b superimposes the R_{work} in dotted lines (full dataset) in comparison with the R_{free} where the fit is performed on 95% of the dataset and the quality assessment is performed on the remaining 5% (dashed line). Thus R_{work} is always expected to be

better than R_{free} . While the R_{work} for the initial dataset looks much better than the sparsified version, the degradation is very limited for the R_{free} , especially between the initial and the sparsified dataset at 1σ . The same is observed for the $CC_{1/2}$ in figure 8c: the initial dataset shows a clear degradation between the *work* and the *free* version, but this degradation is less important for the sparsified version. The $CC_{1/2}^{free}$ indicators hardly degrades between the initial dataset and the two sparsified ones, confirming the ability to solve the protein structure from sparse data.

3.6. Conclusion on sparsification

The sparsification algorithm presented here is generally applicable to any kind of single crystal diffraction experiment where the background is isotropic. This excludes notably diffuse scattering experiment but it is generally applicable to small molecules crystallography and macro-molecular crystallography and even serial crystallography, when the images are nicely centro-symmetric and shadows are properly masked out. The additional compression offered by sparsification is especially interesting for serial crystallography where datasets of millions of frames are collected per experiment. As any lossy compression, the user has still the responsibility of choosing wisely the threshold level as it will limit later-on the quality of the results one may extract from those data. This is of crucial importance for macromolecular crystallography where valuable information is still present in reflection with SNR less than one (Karplus & Diederichs, 2012). We have demonstrated that a sparsification at 0.8σ , which has an actual compression of $2.0\times$ on Eiger data ($2.6\times$ for Jungfrau data), preserves nicely the electron density map and is hardly distinguishable from the uncompressed data. The sparsification at 1.0σ , offering a $2.6\times$ compression on Eiger data ($3.2\times$ for Jungfrau), preserves enough signal to refine a protein in with very limited degradation of the R_{free} and $CC_{1/2}^{free}$ quality indicators. The Jungfrau data show systematically larger

compression rate for sparsification because data was collected with 485 ADU/photon (and is tunable) while Eiger detector always operate at 1 ADU/photon (i.e. it has less noise).

In comparison with ROIBIN-SZ (Underwood *et al.*, 2023), which preserves all pixels in the neighborhood of identified peaks and stores the background heavily binned, the sparsification presented here does not require a complete peak-search thus, it is simpler, with fewer parameters to be tuned, and is able to save any pixel which is intense enough. Nevertheless, peak-finding is of crucial important for identifying Bragg-peaks, one of the first stage of any data reduction pipeline.

4. Application to peak-finding for serial crystallography

A classical way of pre-processing serial-crystallography data is to shrink the amount of data by sieving-out empty or bad frames, only keeping the frames which deserve processing. This is the role of the "veto-algorithm".

The sigma clipping algorithm provides us with the background (average and deviation) and is used to pick pixels which are likely to be part of Bragg peaks, like peakfinder8 (Barty *et al.*, 2014) does. For this, several additional checks are performed on a local neighbourhood which is a small square patch (typically 3x3 or 5x5 pixels, user defined):

- Is the considered pixel the maximum of the local neighborhood ?
- Are enough pixels of the local neighborhood satisfying the SNR condition? (user defined parameter)

For each peak, the coordinates of the centroid, the sum of data and its propagated deviation are recorded and reported. Those peak-position are saved into a HDF5 file (represented figure 9) following the CXI format (Maia, 2012) which can be read from CrystFEL (White *et al.*, 2012). CrystFEL allows to swap peak-picking algo-

rithms (Zaefferer, 2000; Barty *et al.*, 2014; Hadian-Jazi *et al.*, 2021) and indexing tools (Kabsch, 2010; Powell *et al.*, 2013; Ginn *et al.*, 2016; Gevorkov *et al.*, 2019; Gevorkov *et al.*, 2020).

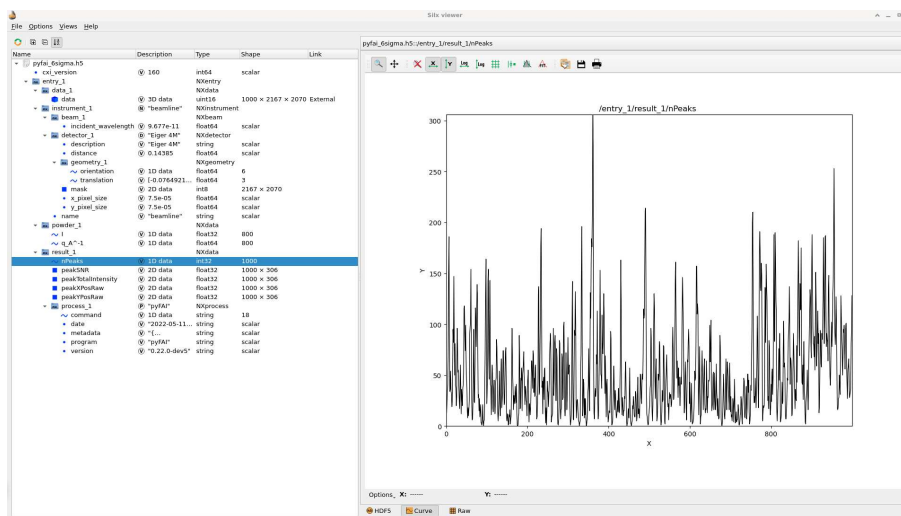


Fig. 9. Peak-picking CXI-file produced by pyFAI and visualized with the *silx viewer* (Vincent *et al.*, 2021). The left-hand side contains the HDF5 tree structure while the right-hand side presents the default plot with the number of peaks found per frame.

The serial crystallography beamline at the ESRF (ID29) uses a LImA2 monitor (Debionne *et al.*, 2022a) as visualization tool. It is inspired from NanoPeakCell (Coquelle *et al.*, 2015) for online visualization and feeds back information to check if peaks found correspond actually to the crystal lattice expected for the sample. We will first compare the peak-picking algorithm with some reference implementation on a single frame before evaluating the quality of the picked points on a serial-crystallography dataset.

4.1. Comparison of picked peaks

Figure 10 presents the comparison between the original *peakfinder8* described in Barty *et al.* (2014), interfaced in Python via OnDA (Mariani *et al.*, 2016) and the version implemented into pyFAI on the same Pilatus 6M image, already used in figure

2.

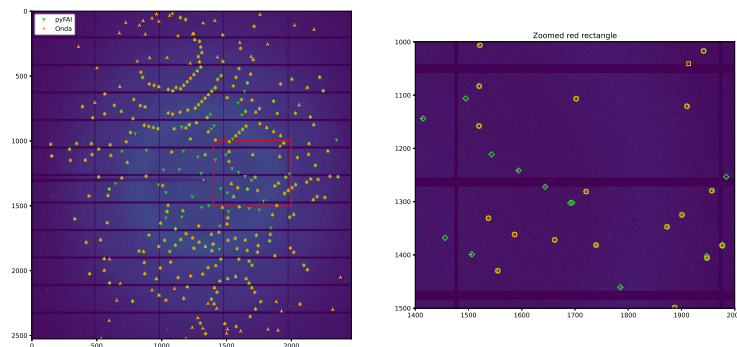


Fig. 10. Comparison of the reference *peakfinder8* interfaced with Onda (in orange, execution time 300ms) and the version from pyFAI (in green, execution time 10ms) on top of a Pilatus 6M diffraction frame of an insulin crystal. The subplot on the right is a close-up to the red rectangle.

In figure 10, most peaks found by both implementation match and correspond to Bragg reflections, the close-up on the right allows to visualize Bragg spots in the image. There are more green peaks (found by pyFAI) closer to the beam center while more orange peaks (found by Onda) are located in the outer-shell. This plot was made with a minimum SNR of 3 and a noise level of 1 since the Pilatus detector is Poissonian. Peaks were registered if four pixels meet the SNR criterion in a 5x5 pixels patch around the peak. Those parameters have been tuned to obtain a comparable number of peaks with both implementations: 290 with pyFAI and 293 with Onda. The similarity of those figures allows a direct comparison of peaks found per resolution shell, histograms which are plotted in figure 11.

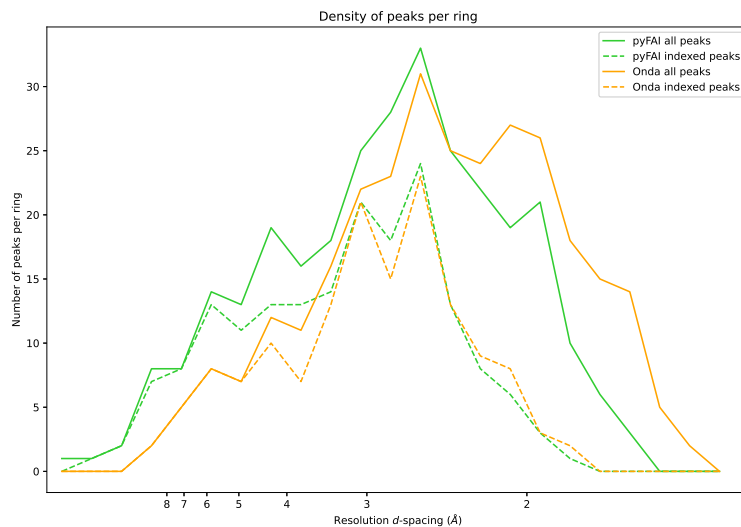


Fig. 11. Number of peaks found in the different resolution shells for the *peakfinder* implemented in pyFAI and in Onda. The width of each radial-shell is two inverse nanometer in q -space. The dashed lines represents the number of those peaks which have successfully been indexed, i.e. located at less than two pixels away from an expected reflection (calculated by *xgandalf*).

The figure 11 presents the histogram of q -values (modulus of the scattering vector) of the peaks found with different methods, i.e. the number of peaks having a given q -value. For readability, those histograms (bin width: 2nm^{-1}) have been represented as plots with the horizontal axis labeled in d-spacing ($q = 2\pi/d$). The analysis of those histograms confirms that the implementation in pyFAI is getting more points closer to the beam-center while the reference implementation is picking more point at larger q -values. This is likely due to the curvature of the Debye-Scherrer ring: the original version of *peakfinder8* evaluates the variance in a neighborhood defined by some radius around the point of interest and the variance is higher close to the beam-center due to the important curvature of those rings. On the opposite, pyFAI knows about this curvature and measures the variance along the ring. Points picked by Onda at larger q -values do not look like Bragg-peaks but this could be a side-effect of the parameter

tuning to get the same number of peaks for both algorithms.

The same figure presents with dashed lines the number of peaks which are coincident (within 2 pixels) with expected reflections (after indexing using *xgandalf*). It demonstrates that the additional peaks found by pyFAI in the inner-shell ($d > 3\text{\AA}$) are consistent with Bragg-peaks, thus valuable for indexing, while the additional peaks found by Onda in the outer shell ($d < 2\text{\AA}$) are not Bragg-peaks, and thus are detrimental.

A word on performances: the Python binding in Onda to the peakfinder8 algorithm from Cheetah runs in 180ms on a high-end server CPU (AMD Epyc 7262) and 300ms on a workstation (Intel Xeon E5-1650 v4). The version available in pyFAI was designed in OpenCL (Khronos, 2008; Stone *et al.*, 2010; Klöckner *et al.*, 2012) and runs best on GPUs: 30ms on a AMD Vega 56 and in 10 ms on a Nvidia RTX A5000.

4.2. Quality of the peakfinder on serial crystallography data

The quality of peaks extracted with this algorithm is evaluated on a serial-crystallography dataset. A subset of 1000 frames of the dataset used in section 3.4 was used as a probe and was indexed with the *indexamajig* tool from CrystFEL. Since all frames show Bragg-peaks (figure 9) the number of indexed frames can be seen as a quality indicator of the peak-picking algorithm used, when all other parameter remain unchanged. The indexation was performed with the *xgandalf* algorithm (Gevorkov *et al.*, 2019) with default settings from CrystFEL v0.10.1 and was provided with the cell parameter: tetragonal $a = b = 78.77\text{\AA}$; $c = 39.04\text{\AA}$; $\alpha = \beta = \gamma = 90^\circ$. The table 3 compares the number of frames properly indexed with the different picking algorithms available in CrystFEL and with the algorithm presented here.

Table 3. Indexation rate obtained with XGANDALF (Gevorkov et al., 2019) from peak positions extracted with different picking algorithms available from CrystFEL (White et al., 2012) on a subset of 1000 frames of microcrystals of Lysozyme (HEWL+Ga) collected with an Eiger 4M at ESRF-ID30A3.

Peak-picking method	XGANDALF (default)		XGANDALF (fast)	
	Index. rate	run-time	Index. rate	run-time
Zaef (Zaefferer, 2000)	10.0%	2178 s	10.0%	430 s
PeakFinder8 (Barty et al., 2014)	49.5%	10397 s	48.5%	1757 s
PeakFinder9 (Gevorkov et al., 2024)	44.2%	8328 s	43.5%	1436 s
RobustPF (Hadian-Jazi et al., 2021)	22.4%	6314 s	21.2%	1628 s
pyFAI (this contribution)	49.7%	9325 s	49.2%	1595 s

Since the Eiger detector is a counting detector, the global threshold for algorithms *zaef* and *peakfinder8* had to be lowered (to 50, which is the maximum of the background signal on any frame) and the the default SNR value was used for *zaef*, *peakfinder8*. The same SNR value of five was used for *pyFAI*. Default parameters were used for *PeakFinder9* and *RobustPeakFinder* algorithms. The reported run-time correspond to the execution time on a single core of an Intel Xeon Gold 6134.

The indexing rate obtained with the algorithm from pyFAI is in par with the reference implementations like *peakfinder8* or *peakfinder9* available from CrystFEL. Since time is mostly spent in indexing, sets of peaks which are simpler to index present lower run-time as fewer retry were needed. Retries can be deactivated but they increase significantly the success of indexing. Table 3 presents also the indexing performances when using the option *-xgandalf-fast-execution* from *indexamajig*, which is five to six times faster and exhibits a limited degradation of the indexing rate. The peak extraction in pyFAI is about as fast as the sparsification, so it can be used online to perform the pre-analysis and provide peaks to NanoPeakCell. Nevertheless the executable ‘peakfinder’ available from pyFAI (offline tool) has a total execution time which is much larger: about 30 s for 1000 frames, most of which is spent in reading and writing the different HDF5 files.

4.3. Peak count as veto algorithm

Since the background extraction and peak finding are performed in real-time on the serial-crystallography beamline ID29 at ESRF, the information about the number of Bragg-spots can be used to assess the quality of each individual image and the acquisition system can decide to discard the frame depending on the number of peaks and a live-adjustable threshold.

Since the beginning of operation of ESRF-ID29, in 2022, the beamline operated with the *veto* algorithm deactivated, the number of peaks found was just recorded for future exploration. The figure 12 presents the indexation rate of "hit" (in blue) and "non-hit" frames (in orange) when changing the threshold for the minimum peaks-count per frame. The expected compression rate is displayed in green. The dataset consists of 80000 lysozyme micro-crystals between two mylar films, raster scanned with a x-ray beam of 11.56 keV at ESRF-ID29. The offline analysis has been performed with CrystFEL (v0.11.0) using 'peakfinder8' and 'xgandalf' as indexer.

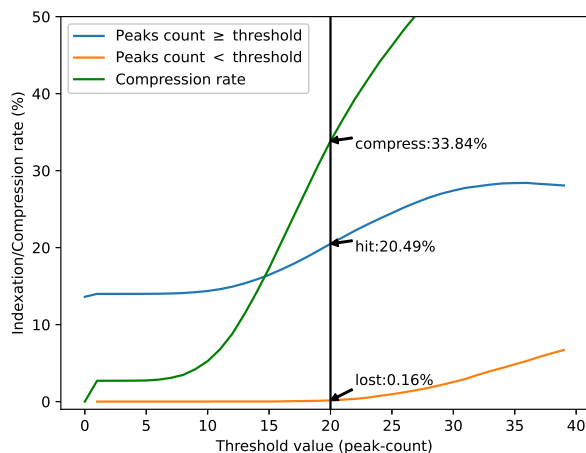


Fig. 12. Indexation rate of frames which would be considered as "hit" or "non-hit" as function of the peak-count threshold. The green curve presents the disk space which could have been saved. The sample dataset consists of 80000 frames of lysozyme micro-crystals indexed off-line with Xgandalf in CrystFEL.

Since the JUNGFRÄU is an integrating detector, the detector has relatively more background noise than a photon counting detector: here, data were saved with 8 ADU/photon, making the compression rate of hit and non-hit frames similar. In this example, discarding frames with less than 20 peaks would have allowed to save one third of the network bandwidth and disk space, losing only 0.16% of frames index-able. The indexation rate of actually recorded frames also increased to 20%. It is noticeable the hit-rate was especially high in this experiment with 84% of frames showing diffraction peaks. One would not expect a user-experiment to have such high concentration of crystals and in normal conditions, the expected compression ratio should be higher. The *veto* algorithm having proved its robustness, it is now activated for most experiments, thanks to graphical helper tools to assess the optimal threshold during the experiment and allowing to set those parameters on the fly for real-time processing.

4.4. Limitations

There is a strong sensitivity of the indexation rate with data from the JUNGFRÄU detector, related to the description of the detector in CrystFEL and in pyFAI. The JUNGFRÄU 4M detector is built from 8 modules manually assembled and exhibits some residual misalignment, on the order of a few pixels and miss-behaving pixels. This misalignment is much larger than what is commonly encountered with Eiger detectors from Dectris, where misalignment is usually less than one pixel in size (75 μm). On Eiger detector data, where the detector is defined as a single rigid module, the indexation rate is fairly independent of the peak-picking algorithm as described in 4.2. This means the peak positions provided by the veto-algorithm are suitable for indexing and this saves the reading of the complete frame if it cannot be indexed. With data from the JUNGFRÄU detector, the indexing rate was lower than with certain peak-finders algorithms integrated into CrystFEL. Work is ongoing to convert the

geometry description from pyFAI, where every pixel is independent, to the geometry used in CrystFEL, and vice-versa. We found this difference of indexation was related to the description of the mask and of pixel position in different software. Until this issue is addressed, the safest is to re-extract peak positions for indexing using one of CrystFEL's provided peak-finding algorithms.

5. Conclusion

Background analysis of single crystal diffraction images can be implemented efficiently using iterative azimuthal integration and allows the separation of the signal originating from Bragg-peaks from isotropic background.

A lossy compression algorithm for diffraction frames, called sparsification, has been built on top of this signal separation, with the average background saved on the one side and the position and the intensity of most intense pixels (probably belonging to peaks) on the the other. The quality of the compression has been demonstrated on macro-molecular rotational and serial crystallography data. The degradation of the signal has been monitored after a compression-decompression cycle, both in reciprocal space using crystallographic quality indicators and after Fourier transform in direct space where the number of residues placed automatically was checked. The degradation of the signal was also monitored as function of the threshold and sparsification at $0.8\sigma - 1.0\sigma$ still enables to reconstruct the molecular structure of proteins, both with Eiger and JUNGFR AU detectors.

The second application is a peak-finder for serial crystallography which locates peak positions in real time and can be used as veto algorithm to discard images without (enough) diffraction peaks. The peaks picked have been evaluated against state of the art peak-finder algorithms like *peakfinder8* and the results were comparable in quality while much faster thanks to the usage of GPU. This *veto*-algorithm is now used in

production at the ESRF serial crystallography beamline (ID29) to save storage space.

One of the strength of this peak-finer algorithm is that it is optimized to work on GPU and thus ideally suited to be coupled with the next generation of crystal indexer (Gasparotto *et al.*, 2024) which is running on the same kind of hardware, allowing to save two memory-transfer per frame and maybe achieve real-time integration of serial crystallography data.

Acknowledgements: The authors would like to thank Andy Götz for the management of this project and Vincent Favre-Nicolin for his support. Thanks also to Gavin Vaughan, scientist at Materials beamline at the ESRF, for the constructive discussion about sigma-clipping versus median filtering. We would like to thank also Jonathan P. Wright and Carlotta Giacobbe for offering us the ability to test those algorithms on small molecule data and validate the concept of sparsification on the ID11 beamline at the ESRF. Pierre Paleo and Jerome Lesaint are also acknowledged for the fruitful discussions on numerical methods developed in this document.

References

- Afonine, P. V., Grosse-Kunstleve, R. W., Echols, N., Headd, J. J., Moriarty, N. W., Mustyakimov, M., Terwilliger, T. C., Urzhumtsev, A., Zwart, P. H. & Adams, P. D. (2012). *Acta Crystallographica Section D*, **68**(4), 352–367.
<https://doi.org/10.1107/S0907444912001308>
- Barty, A., Kirian, R. A., Maia, F. R. N. C., Hantke, M., Yoon, C. H., White, T. A. & Chapman, H. (2014). *Journal of Applied Crystallography*, **47**(3), 1118–1131.
<https://doi.org/10.1107/S1600576714007626>
- Blelloch, G. E. (1996). *Commun. ACM*, **39**(3), 85–97.
<https://doi.org/10.1145/227234.227246>
- Blobel, V. & Kleinwort, C., (2002). A new method for the high-precision alignment of track detectors.
<https://arxiv.org/abs/hep-ex/0208021>
- Bordet, P., Kergourlay, F., Pinto, A., Blanc, N. & Martinetto, P. (2021). *J. Anal. At. Spectrom.* **36**, 1724–1734.
<http://dx.doi.org/10.1039/D1JA00143D>
- Boutet, S., Lomb, L., Williams, G. J., Barends, T. R. M., Aquila, A., Doak, R. B., Weierstall, U., DePonte, D. P., Steinbrener, J., Shoeman, R. L., Messerschmidt, M., Barty, A., White, T. A., Kassemeyer, S., Kirian, R. A., Seibert, M. M., Montanez, P. A., Kenney, C., Herbst, R., Hart, P., Pines, J., Haller, G., Gruner, S. M., Philipp, H. T., Tate, M. W., Hromalik, M., Koerner, L. J., van Bakel, N., Morse, J., Ghonsalves, W., Arnlund, D., Bogan, M. J., Caleman, C., Fromme, R., Hampton, C. Y., Hunter, M. S., Johansson, L. C., Katona, G., Kupitz, C., Liang, M., Martin, A. V., Nass, K., Redecke, L., Stellato, F., Timneanu, N., Wang, D., Zatsepin, N. A., Schafer, D., DeFevers, J., Neutze, R., Fromme, P., Spence, J. C. H., Chapman, H. N. & Schlichting, I. (2012). *Science*, **337**(6092), 362–364.
<https://www.science.org/doi/abs/10.1126/science.1217737>

- Broennimann, C., Eikenberry, E. F., Henrich, B., Horisberger, R., Huelsen, G., Pohl, E., Schmitt, B., Schulze-Briesse, C., Suzuki, M., Tomizaki, T., Toyokawa, H. & Wagner, A. (2006). *Journal of Synchrotron Radiation*, **13**(2), 120–130.
<https://doi.org/10.1107/S0909049505038665>
- Brünger, A. T. (1992). *Nature*, **355**, 472–475.
<https://doi.org/10.1038/355472a>
- Casanas, A., Warshamanage, R., Finke, A. D., Panepucci, E., Olieric, V., Nöll, A., Tampé, R., Brandstetter, S., Förster, A., Mueller, M., Schulze-Briesse, C., Bunk, O. & Wang, M. (2016). *Acta Crystallographica Section D*, **72**(9), 1036–1048.
<https://doi.org/10.1107/S2059798316012304>
- Chaize, J. *et al.* (2018). In *Proc. of International Conference on Accelerator and Large Experimental Control Systems (ICALEPCS'17), Barcelona, Spain, 8-13 October 2017*, no. 16 in International Conference on Accelerator and Large Experimental Control Systems, pp. 2010–2015. Geneva, Switzerland: JACoW. <https://doi.org/10.18429/JACoW-ICALEPCS2017-FRAPL07>.
<http://jacow.org/icalepcs2017/papers/frapl07.pdf>
- Chapman, H. N., Fromme, P., Barty, A., White, T. A., Kirian, R. A., Aquila, A., Hunter, M. S., Schulz, J., DePonte, D. P., Weierstall, U., Doak, R. B., Maia, F. R. N. C., Martin, A. V., Schlichting, I., Lomb, L., Coppola, N., Shoeman, R. L., Epp, S. W., Hartmann, R., Rolles, D., Rudenko, A., Foucar, L., Kimmel, N., Weidenspointner, G., Holl, P., Liang, M., Barthelmess, M., Caleman, C., Boutet, S., Bogan, M. J., Krzywinski, J., Bostedt, C., Bajt, S., Gumprecht, L., Rudek, B., Erk, B., Schmidt, C., Homke, A., Reich, C., Pietschner, D., Struder, L., Hauser, G., Gorke, H., Ullrich, J., Herrmann, S., Schaller, G., Schopper, F., Soltau, H., Kühnel, K.-U., Messerschmidt, M., Bozek, J. D., Hau-Riege, S. P., Frank, M., Hampton, C. Y., Sierra, R. G., Starodub, D., Williams, G. J., Hajdu, J., Timneanu, N., Seibert, M. M., Andreasson, J., Rocker, A., Jonsson, O., Svenda, M., Stern, S., Nass, K., Andritschke, R., Schröter, C.-D., Krasniqi, F., Bott, M., Schmidt, K. E., Wang, X., Grotjohann, I., Holton, J. M., Barends, T. R. M., Neutze, R., Marchesini, S., Fromme, R., Schorb, S., Rupp, D., Adolph, M., Gorkhover, T., Andersson, I., Hirsemann, H., Potdevin, G., Graafsma, H., Nilsson, B., Spence, J. C. H. & DESY (2011). *Nature*, **470**, 73–77.
<https://bib-pubdb1.desy.de/record/87161>
- Coquelle, N., Brewster, A. S., Kapp, U., Shilova, A., Weinhausen, B., Burghammer, M. & Colletier, J.-P. (2015). *Acta Crystallographica Section D*, **71**(5), 1184–1196.
<https://doi.org/10.1107/S1399004715004514>
- Debionne, S., Homs, A. & Claustre, L., (2022a). Library for IMage Acquisition version 2.
<https://gitlab.esrf.fr/limagroup/lima2>
- Debionne, S., Homs, A., Claustre, L., Kieffer, J., De Sanctis, D., Santoni, G., Goetz, A. & Meyer, J. (2022b). In *Proceedings of the 14th international conference on Synchrotron Radiation Instrumentation (SRI2021)*.
<https://indico.desy.de/event/27430/abstracts/>
- Dectris, (2014).
<https://www.dectris.com/support/downloads/datasets/>
- Diederichs, K. & Karplus, P. A. (1997). *Nature Structural Biology*, **4**(4), 269–275.
- Diederichs, K. & Wang, M. (2017). *Serial Synchrotron X-Ray Crystallography (SSX)*, pp. 239–272. New York, NY: Springer New York.
https://doi.org/10.1007/978-1-4939-7000-1_10
- Fang, F., Wang, T., Wu, S. & Zhang, G. (2020). *Information Sciences*, **514**, 56–70.
<https://www.sciencedirect.com/science/article/pii/S0020025519311090>
- Galchenkova, M., Tolstikova, A., Klopprogge, B., Sprenger, J., Oberthuer, D., Brehm, W., White, T. A., Barty, A., Chapman, H. N. & Yefanov, O. (2024). *IUCrJ*, **11**(2), 190–201.
<https://doi.org/10.1107/S205225252400054X>
- Gasparotto, P., Barba, L., Stadler, H.-C., Assmann, G., Mendonça, H., Ashton, A. W., Janousch, M., Leonarski, F. & Béjar, B. (2024). *Journal of Applied Crystallography*, **57**(4), 931–944.
<https://doi.org/10.1107/S1600576724003182>

- Gati, C., Bourenkov, G., Klinge, M., Rehders, D., Stellato, F., Oberthür, D., Yefanov, O., Sommer, B. P., Mogk, S., Duszynski, M., Betzel, C., Schneider, T. R., Chapman, H. N. & Redecke, L. (2014). *IUCrJ*, **1**(2), 87–94.
<https://doi.org/10.1107/S2052252513033939>
- Gevorkov, Y., Barty, A., Brehm, W., White, T. A., Tolstikova, A., Wiedorn, M. O., Meents, A., Grigat, R.-R., Chapman, H. N. & Yefanov, O. (2020). *Acta Crystallographica Section A*, **76**(2), 121–131.
<https://doi.org/10.1107/S2053273319015559>
- Gevorkov, Y., Galchenkova, M., Mariani, V., Barty, A., White, T. A., Chapman, H. N. & Yefanov, O. (2024). *Crystals*, **14**(2).
<https://www.mdpi.com/2073-4352/14/2/164>
- Gevorkov, Y., Yefanov, O., Barty, A., White, T. A., Mariani, V., Brehm, W., Tolstikova, A., Grigat, R.-R. & Chapman, H. N. (2019). *Acta Crystallographica Section A*, **75**(5), 694–704.
<https://doi.org/10.1107/S2053273319010593>
- Ginn, H. M., Roedig, P., Kuo, A., Evans, G., Sauter, N. K., Ernst, O. P., Meents, A., Mueller-Werkmeister, H., Miller, R. J. D. & Stuart, D. I. (2016). *Acta Crystallographica Section D*, **72**(8), 956–965.
<https://doi.org/10.1107/S2059798316010706>
- Grieco, A., Boneta, S., Gavira, J. A., Pey, A. L., Basu, S., Orlans, J., Sanctis, D. d., Medina, M. & Martin-Garcia, J. M. (2024). *Protein Science*, **33**(4), e4957.
<https://onlinelibrary.wiley.com/doi/abs/10.1002/pro.4957>
- Hadian-Jazi, M., Sadri, A., Barty, A., Yefanov, O., Galchenkova, M., Oberthuer, D., Komadina, D., Brehm, W., Kirkwood, H., Mills, G., de Wijn, R., Letrun, R., Kloos, M., Vakili, M., Gelisio, L., Darmanin, C., Mancuso, A. P., Chapman, H. N. & Abbey, B. (2021). *Journal of Applied Crystallography*, **54**(5), 1360–1378.
<https://doi.org/10.1107/S1600576721007317>
- Hammersley, A. P., Svensson, S. O., Hanfland, M., Fitch, A. N. & Hausermann, D. (1996). *High Press. Res.* **14**(4-6), 235–248.
- Joldes, M., Muller, J.-M. & Popescu, V. (2017). *ACM Trans. Math. Softw.* **44**(2).
<https://doi.org/10.1145/3121432>
- Kabsch, W. (2010). *Acta Crystallographica Section D*, **66**(2), 133–144.
<https://doi.org/10.1107/S0907444909047374>
- Karplus, P. A. & Diederichs, K. (2012). *Science*, **336**(6084), 1030–1033.
<https://www.science.org/doi/abs/10.1126/science.1218231>
- Khronos, (2008). The open standard for parallel programming of heterogeneous systems.
<http://www.khronos.org/OpenGL>
- Kieffer, J. & Ashiotis, G. (2014). In *PROC. OF THE 7th EUR. CONF. ON PYTHON IN SCIENCE (EUROSCIPY 2014)*.
<http://arxiv.org/pdf/1412.6367.pdf>
- Kieffer, J., Valls, V., Blanc, N. & Hennig, C. (2020). *Journal of Synchrotron Radiation*, **27**(2), 558–566.
<https://doi.org/10.1107/S1600577520000776>
- Kieffer, J. & Wright, J. (2013). *Powder Diffraction*, **28**(S2), S339–S350.
- Kleinwort, C., (2021-2024). Millepede-ii.
<https://gitlab.desy.de/claus.kleinwort/millepede-ii>
- Klößner, A., Pinto, N., Lee, Y., Catanzaro, B., Ivanov, P. & Fasih, A. (2012). *Parallel Computing*, **38**(3), 157–174.
- Knudsen, E. B., Sørensen, H. O., Wright, J. P., Goret, G. & Kieffer, J. (2013). *J. Appl. Cryst.* **46**, 537–539.
- Könnecke, M., Akeroyd, F. A., Bernstein, H. J., Brewster, A. S., Campbell, S. I., Clausen, B., Cottrell, S., Hoffmann, J. U., Jemian, P. R., Männicke, D., Osborn, R., Peterson, P. F., Richter, T., Suzuki, J., Watts, B., Wintersberger, E. & Wuttke, J. (2015). *Journal of Applied Crystallography*, **48**(1), 301–305.
<https://doi.org/10.1107/S1600576714027575>

- Leonarski, F., Mozzanica, A., Brückner, M., Lopez-Cuenca, C., Redford, S., Sala, L., Babic, A., Billich, H., Bunk, O., Schmitt, B. & Wang, M. (2020). *Structural Dynamics*, **7**(1), 014305.
<https://doi.org/10.1063/1.5143480>
- Maia, F. (2012). *Nature Methods*, **9**, 854–855.
- Maples, M. P., Reichart, D. E., Konz, N. C., Berger, T. A., Trotter, A. S., Martin, J. R., Dutton, D. A., Paggen, M. L., Joyner, R. E. & Salemi, C. P. (2018). *The Astrophysical Journal Supplement Series*, **238**(1), 2.
<https://doi.org/10.3847/1538-4365/aad23d>
- Mariani, V., Morgan, A., Yoon, C. H., Lane, T. J., White, T. A., O’Grady, C., Kuhn, M., Aplin, S., Koglin, J., Barty, A. & Chapman, H. N. (2016). *Journal of Applied Crystallography*, **49**(3), 1073–1080.
<https://doi.org/10.1107/S1600576716007469>
- Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Höfer, C., Halpern, M., Hanna, D., Hincks, A., Hinshaw, G., Parra, J., Newburgh, L., Shaw, J. & Vanderlinde, K. (2015). *Astronomy and Computing*, **12**, 181–190.
<https://www.sciencedirect.com/science/article/pii/S2213133715000694>
- Mozzanica, A., Bergamaschi, A., Brueckner, M., Cartier, S., Dinapoli, R., Greiffenberg, D., Jungmann-Smith, J., Maliakal, D., Mezza, D., Ramilli, M., Ruder, C., Schaedler, L., Schmitt, B., Shi, X. & Tinti, G. (2016). *Journal of Instrumentation*, **11**(02), C02047–C02047.
<https://doi.org/10.1088/1748-0221/11/02/c02047>
- Nogly, P., James, D., Wang, D., White, T. A., Zatsepin, N., Shilova, A., Nelson, G., Liu, H., Johansson, L., Heymann, M., Jaeger, K., Metz, M., Wickstrand, C., Wu, W., Båth, P., Bernsten, P., Oberthuer, D., Panneels, V., Cherezov, V., Chapman, H., Schertler, G., Neutze, R., Spence, J., Moraes, I., Burghammer, M., Standfuss, J. & Weierstall, U. (2015). *IUCrJ*, **2**(2), 168–176.
<https://doi.org/10.1107/S2052252514026487>
- Owen, R. L., Axford, D., Sherrell, D. A., Kuo, A., Ernst, O. P., Schulz, E. C., Miller, R. J. D. & Mueller-Werkmeister, H. M. (2017). *Acta Crystallographica Section D*, **73**(4), 373–378.
<https://doi.org/10.1107/S2059798317002996>
- Petitdemange, S. *et al.* (2018). In *Proc. of International Conference on Accelerator and Large Experimental Control Systems (ICALPCS’17), Barcelona, Spain, 8-13 October 2017*, no. 16 in International Conference on Accelerator and Large Experimental Control Systems, pp. 886–890. Geneva, Switzerland: JACoW. <https://doi.org/10.18429/JACoW-ICALPCS2017-TUPHA194>.
<http://jacow.org/icalpcs2017/papers/tupha194.pdf>
- Powell, H. R., Johnson, O. & Leslie, A. G. W. (2013). *Acta Crystallographica Section D*, **69**(7), 1195–1203.
<https://doi.org/10.1107/S0907444912048524>
- Real, L. C. V. & de Bayser, M., (2021). User-defined functions for hdf5.
<https://arxiv.org/abs/2109.11709>
- Rigaku Oxford Diffraction, (2015). CrysAlispro software system.
<https://www.rigaku.com/fr/products/smc/crystalis>
- Schubert, E. & Gertz, M. (2018). In *Proceedings of the 30th International Conference on Scientific and Statistical Database Management, SSDBM ’18*. New York, NY, USA: Association for Computing Machinery.
<https://doi.org/10.1145/3221269.3223036>
- Sivia, D. S. & Skilling, J. (2006). *Data Analysis - A Bayesian Tutorial*. Oxford Science Publications. Oxford University Press, 2nd ed.
- Stone, J. E., Gohara, D. & Shi, G. (2010). *Computing in Science and Engineering*, **12**(3), 66–73.
- The HDF Group, (2000-2021). Hierarchical data format version 5.
<http://www.hdfgroup.org/HDF5>
- Toledo, S. (1997). *IBM Journal of Research and Development*, **41**(6), 711–725.

- Underwood, R., Yoon, C., Gok, A., Di, S. & Cappello, F. (2023). *Synchrotron Radiation News*, **36**(4), 17–22.
<https://doi.org/10.1080/08940886.2023.2245722>
- Vincent, T., Valls, V., Payno, H., Kieffer, J., Solé, V. A., Paleo, P., Naudet, D., de Nolf, W., Knobel, P., Garriga, J., Retegan, M., Rovezzi, M., Fangohr, H., Kenter, P., UUSim, Favre-Nicolin, V., Nemoz, C., Picca, F.-E., Caswell, T. A., Bertoldo, J. P. C., Campbell, A., Wright, C. J. C., Communie, G., Kotanski, J., Coutinho, T., N0B0dY, Kim, S.-W., schooft, Farago, T. & linupi, (2021). silx-kit/silx: 1.0.0: 2021/12/06. Zenodo.
<https://doi.org/10.5281/zenodo.5761269>
- White, T. A., Kirian, R. A., Martin, A. V., Aquila, A., Nass, K., Barty, A. & Chapman, H. N. (2012). *Journal of Applied Crystallography*, **45**(2), 335–341.
<https://doi.org/10.1107/S0021889812002312>
- Zaefferer, S. (2000). *Journal of Applied Crystallography*, **33**(1), 10–25.
<https://doi.org/10.1107/S0021889899010894>

Synopsis

Precise background assessment and application to single crystal image compression and serial crystallography data pre-processing.
